

Databases

Lecture 10 - Database Applications

Emily Lucia Antosch

HAW Hamburg

04.04.2026

Contents

1. Introduction	7
2. Repetition	11
ACID - Durability	12
Distributed Transactions	13
Integrity Constraints	14
Integrity Constraints - ECA	15
Excursion Delimiter: Example	16
Types	17
Transition Variables	18
Permissions	19
GRANT and REVOKE	20
3. Database Applications	21

Problems	22
Combining SQL with 3GL	23
Impedance Mismatch	24
Embedded SQL	25
SQL in C	26
SQL in Java	27
Embedded SQL	28
API Calls	29
API Calls - JDBC	30
API Calls - JDBC: Preparation	31
API Calls - JDBC: Preparation	32
API Calls - JDBC: Preparation	33
API Calls - JDBC	34

API Calls - JDBC	35
API Calls - JDBC	36
API Calls - JDBC	37
API Calls - JDBC	38
API Calls - JDBC	39
API Calls - JDBC	40
API Calls - JDBC	41
API Calls - JDBC	42
API Calls - JDBC	43
API Calls - SQL Injection	44
API Calls	45
API Calls	46
API Calls - Transaction Handling	47

API Calls - Error Handling	48
API Calls - Antipatterns	49
API Calls - Antipatterns	50
API Calls - Beyond JDBC	51
Extend SQL	52
Extend SQL - PL/SQL	53
Extend SQL - PL/SQL	54
Extend SQL - PL/SQL	55
Extend SQL - PL/SQL	56
Extend SQL - PL/SQL	57
Extend SQL - PL/SQL	58
Extend SQL - Example	59
Extend SQL - Example	60

Extend SQL - PL/SQL	61
Extend SQL - PL/SQL	62
Extend SQL - Java Stored Procedures	63
4. License Notice	67

1. Introduction

1.1 Where are we right now?

- Last time, we looked at the basics of subqueries and views
- Today, we'll be discussing
 - ▶ what solutions there are for calling SQL from code,
 - ▶ how we can integrate our database with Java and
 - ▶ in what way databases can be used in applications.

1.1 Where are we right now?

1. Introduction
2. Basics
3. SQL
4. Entity-Relationship-Model
5. Relationships
6. Constraints
8. Subqueries & Views
9. Transactions
10. **Database Applications**
11. Integrity, Trigger & Security

1.2 What is the goal of this chapter?

- At the end of this lesson, you should be able to
 - ▶ create a simple app using Java and PostgreSQL.

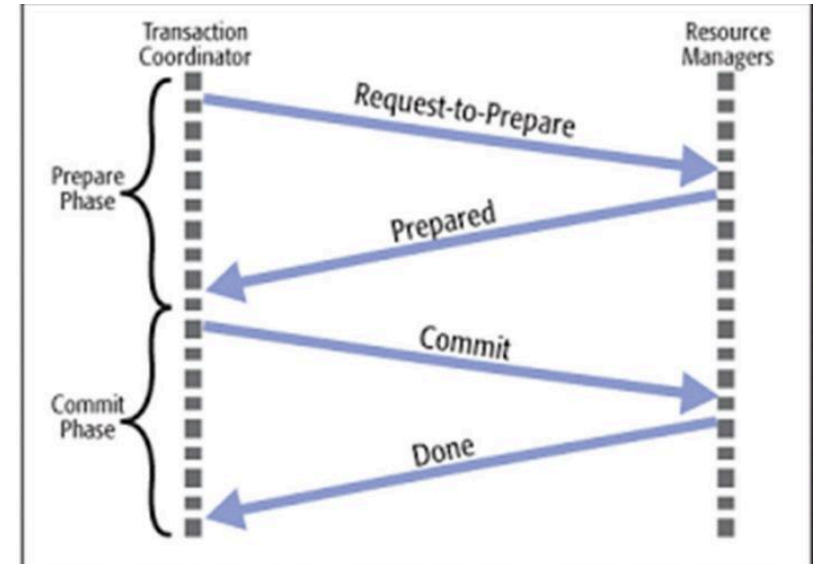
2. Repetition

ACID - Durability

- Once committed, changed data is safe
- Error types
 1. Computer failure
 2. Transaction or system error (constraint violation, $\frac{x}{0}$, blackout, system crash)
 3. Local Errors
 4. Concurrency control enforcement
 5. Disk error (harddisk broken)
 6. Physical problems and catastrophes (fire, earthquake, robbery, ...)

Distributed Transactions

- To ensure interoperability between the participating resource managers the **2-phase commit protocol** is realized
- It defines the final synchronization of different parts of a transaction of a global transaction
- In the first phase the transaction manager asks participating resource managers to announce the results of their local transaction part
- This leads to a global result (commit or rollback) that is then in the second phase announced to the participants



Integrity Constraints

- Static Constraints
 - ▶ Conditions on states
 - ▶ Conditions must be fulfilled before and after operations
 - ▶ Used until now
 - Primary Key
 - Foreign Key
 - UNIQUE, NOT NULL, CHECK
- Dynamic Constraints (**Assertions**)
 - ▶ Integrity conditions that affect multiple tables
 - ▶ Conditions on state transitions

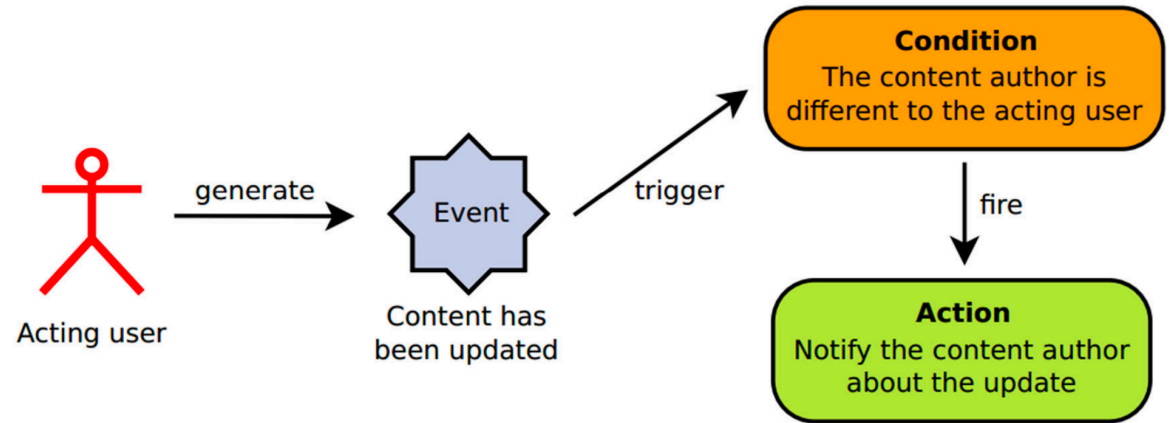


Example

status of order → new →
payed → processing →
shipped

Integrity Constraints - ECA

- ECA rules
 - ▶ on event (E)
 - ▶ under certain conditions (C)
 - ▶ perform actions (A)



Excursion Delimiter: Example

```
1 delimiter |
2 CREATE TRIGGER SALARY_VIOLATION
3 BEFORE INSERT ON EMPLOYEE
4 FOR EACH ROW
5 BEGIN
6     IF NEW.SALARY > (SELECT SALARY
7                     FROM EMPLOYEE
8                     WHERE SSN = NEW.SUPER_SSN )
9     THEN SET NEW.Salary = (SELECT SALARY
10                           FROM EMPLOYEE
11                           WHERE SSN = NEW.SUPER_SSN ) - 1;
12 END IF;
13 END;
14 |
15 delimiter;
```

 SQL

Types

- Time of execution, relative to event
 - ▶ BEFORE
 - ▶ AFTER
- INSTEAD OF
- Statement trigger
 - ▶ Once per statement
 - ▶ Even if no row is affected!
 - ▶ Default trigger type
- Row trigger
- Syntax: FOR EACH ROW

Transition Variables

- Row triggers can access old and new tuples
 - ▶ PostgreSQL
 - `:old` or `old` → NULL for INSERT
 - `:new` or `new` → NULL for DELETE
 - Oracle
 - ▶ NEW and OLD
 - ▶ Before row triggers:
 - Can even modify new!

Permissions

- DBMS are multi-user systems
- You need permissions to do anything with the DB:
 - ▶ login
 - ▶ CREATE table, DROP table, etc.
 - ▶ SELECT
 - ▶ INSERT, UPDATE, DELETE
- Permissions can be GRANTED and REVOKED

GRANT and REVOKE

- Permissions can be GRANTED and REVOKED

```
1 GRANT <privilege_name> ON <object_name>
2 TO { <user_name> | PUBLIC | <role_name> } [ WITH GRANT OPTION ] ;
```

- GRANT

```
1 GRANT SELECT ON tab_a TO user_a;
2 GRANT UPDATE ON tab_b TO user_a;
```

- REVOKE

```
1 REVOKE SELECT ON tab_a FROM user_a;
```

3. Database Applications

Problems

- Cannot solve every problem with SQL
 - ▶ No loops
- Recursion not widely implemented
- Need to query DB out of an application



Idea

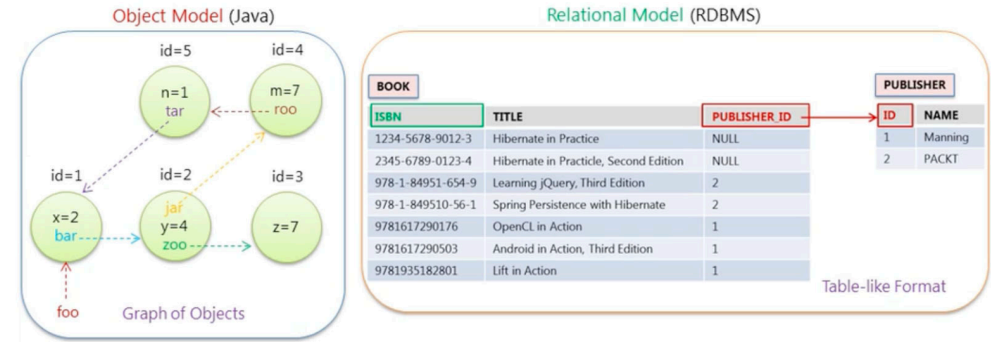
Combination with procedural or object-oriented programming languages (**host languages**)

Combining SQL with 3GL

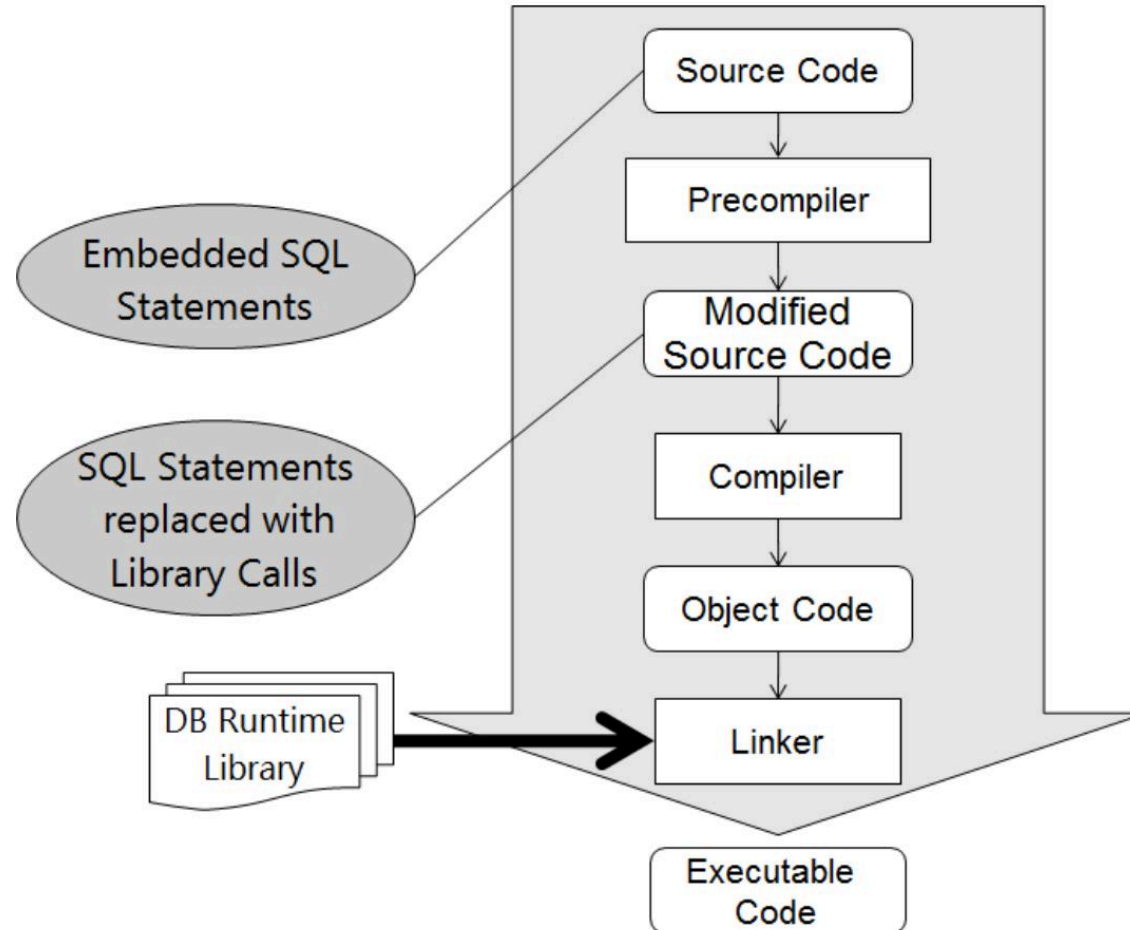
1. **Embed SQL commands into host language**
 - Embedded SQL, SQL/OLB
2. **SQL commands through API calls**
 - SQL: Call Level Interface (CLI)
 - ODBC, JDBC
3. **Extend SQL**
 - SQL: Persistent Stored Modules (SQL/PSM)
 - Oracle: PL/SQL

Impedance Mismatch

- E.g., Object–relational impedance mismatch
 - ▶ Object-oriented concepts, like inheritance in OO, polymorphism in OO,...
 - Data type differences, like pointers in OO,...
 - Structural and integrity differences, like constraints in RM, objects can be composed of other objects in OO, ...
 - Transactional differences, like transactions in RM
 - Manipulative differences, like declarative queries in RM



Embedded SQL



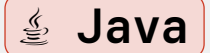
SQL in C

```
1  int main() {
2      exec sql begin declare section;
3      int sv_new_price;
4      int sv_isbn;
5      exec sql end declare section;
6      printf("Please enter ISBN: \n ");
7      scanf("%d", &sv_isbn);
8      printf("Please enter new_price: \n");
9      scanf("%d", &sv_new_price);
10     exec sql update book
11         set price = :sv_new_price
12         where isbn = :sv_isbn;
13 } Shared variables
```



SQL in Java

```
1 int maxSalary, avgSalary;
```



```
2
```

```
3 #sql{
```

```
4 SELECT MAX(SALARY) , AVG(SALARY)
```

```
5     INTO :maxSalary , :avgSalary
```

```
6     FROM EMPLOYEE
```

```
7 };
```

Embedded SQL

- Mainly static SQL
 - ▶ SQL statement is fixed
 - ▶ SQL syntax is checked at (pre-)compile time
- Exchange data with application by **host variables** (:varname)
- Precompilers exist for many languages
 - ▶ C/C++, Java (SQLJ), Ada, Cobol, Fortran, PL1, ...

API Calls

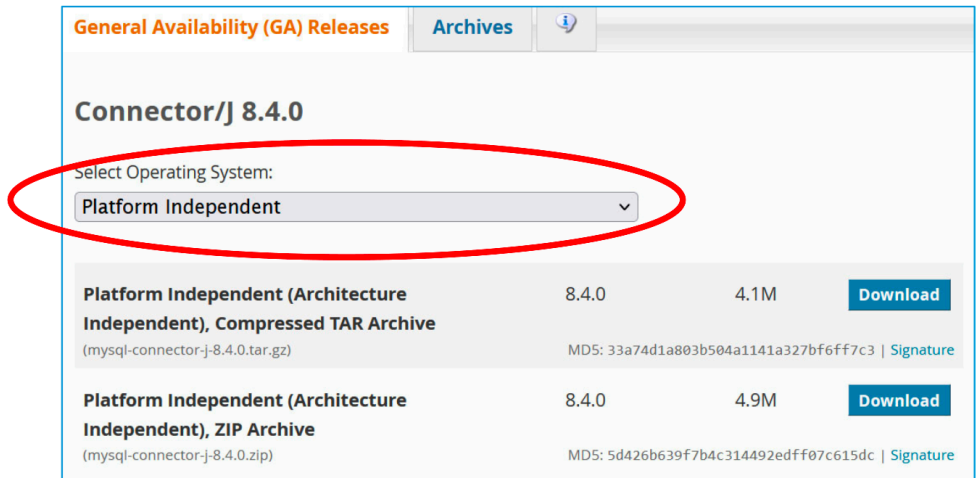
- SQL commands through library/API calls
- Dynamic SQL
 - ▶ Application can dynamically set up the SQL command string
 - ▶ SQL syntax is checked at runtime
- Standard SQL: Call Level Interface (CLI), e.g.,
 - ▶ ODBC (for any language like C,C++,Java, but restricted on MS Windows)
 - ▶ JDBC (for Java, can be used for any platform)
 - ▶ OCI (Oracle Call Interface)

API Calls - JDBC

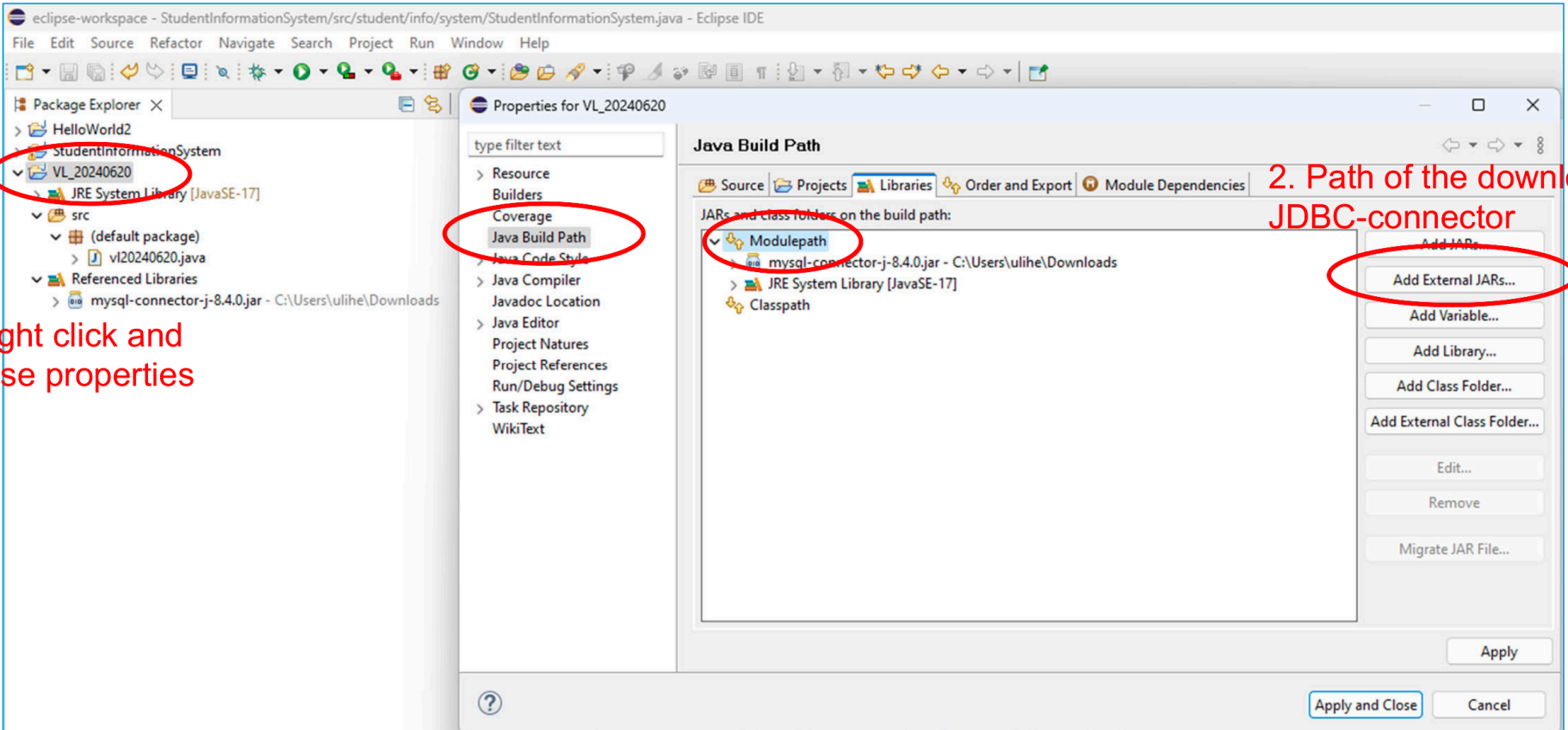
- JDBC: Java Database Connectivity
- Part of Java API
- Typical steps:
 1. Load JDBC driver
 2. Define DB connection URL
 3. Connect to DB
 4. Create command object
 5. Execute command
 6. Process result
 7. Cleanup: Close resources and DB connection

API Calls - JDBC: Preparation

- Download JDBC Connector:
 - ▶ Oracle: <http://java.sun.com/products/jdbc/download.html>
 - ▶ MySQL: <https://dev.mysql.com/downloads/connector/j/>
- Prepare a Java Project, e.g. in Eclipse
- Prepare a MySQL database
- Import the JDBC library



API Calls - JDBC: Preparation



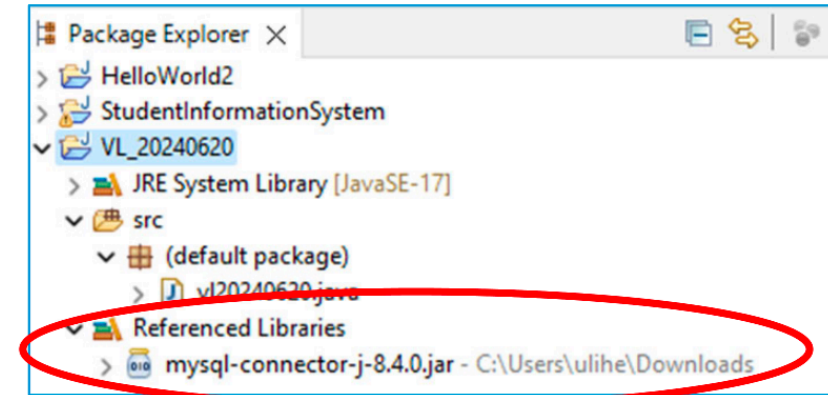
1. Right click and choose properties

2. Path of the downloaded JDBC-connector

The screenshot shows the Eclipse IDE with the 'Properties for VL_20240620' dialog box open. The 'Java Build Path' tab is selected. The 'Modulepath' checkbox is checked, and the 'mysql-connector-j-8.4.0.jar' file is listed. The 'Add External JARs...' button is highlighted. Red annotations indicate the steps: 1. Right click and choose properties (pointing to the project in the Package Explorer) and 2. Path of the downloaded JDBC-connector (pointing to the jar file and the 'Add External JARs...' button).

API Calls - JDBC: Preparation

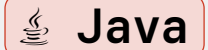
- Download JDBC Connector:
 - ▶ Oracle: <http://java.sun.com/products/jdbc/download.html>
 - ▶ MySQL: <https://dev.mysql.com/downloads/connector/j/>
- Prepare a Java Project, e.g. in Eclipse
- Prepare a MySQL database
- Import the JDBC library



You can see if include was successful

API Calls - JDBC

```
1 Class.forName(driverName);
```



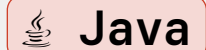
```
2
```

```
3 import driverName;
```



Example

```
1 import java.sql.Connection;
```



```
2 import java.sql.DriverManager;
```

```
3 import java.sql.ResultSet;
```

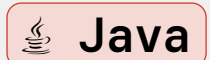
```
4 import java.sql.SQLException;
```

```
5 import java.sql.Statement;
```

API Calls - JDBC

- Connection is defined by an URL
 - ▶ Oracle
 - `jdbc:oracle:thin:@<server>:1521:<dbname>`
 - For example, `Oracle@HAW` (available before the cyber attack):
`jdbc:oracle:thin:@ora14:informatik.haw-hamburg.de:1521:inf14`
 - ▶ MySQL
 - `jdbc:mysql://<server>/<dbname>`
 - For example: `jdbc:mysql://localhost:3306/company`

```
1 private static final String CONN = "jdbc:mysql://  
localhost:3306/company_2024";
```



API Calls - JDBC

- Exkursion: `localhost`
 - ▶ In computer networking, `localhost` is a hostname that refers to the current computer used to access it. It is used to access the network services that are running on the host via the loopback network interface. Using the loopback interface bypasses any local network interface hardware.
 - ▶ The local loopback mechanism may be used to run a network service on a host without requiring a physical network interface, or without making the service accessible from the networks the computer may be connected to. For example, a locally installed website may be accessed from a Web browser by the URL `http://localhost` to display its home page.
 - ▶ The name `localhost` normally resolves to the IPv4 loopback address `127.0.0.1`, and to the IPv6 loopback address `::1`. (Ipv stands for Internet Protocol version)

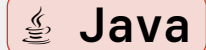
API Calls - JDBC

```
1 Connection conn = DriverManager.getConnection(url, user,  
    psw);
```



Example

```
1 myConn = DriverManager.getConnection(CONN, USER,  
    PASSWORD);
```



- Info about the connection has become available:

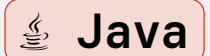
```
1 conn.getMetaData();
```



API Calls - JDBC

- Obtain Statement object

```
1 Statement st = conn.createStatement() ;
```



Also: `prepareStatement()`, `prepareCall()`

API Calls - JDBC

- Execute query

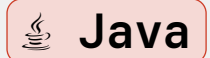
```
1 String query = "SELECT dnumber, dname FROM DEPARTMENT";  
2 // No ";" in query string  
3 ResultSet myRes2 = myStmt2.executeQuery(query);  
4 ResultSet myRes = myStmt.executeQuery("SELECT lname, fname FROM  
  EMPLOYEE");
```

 **Java**

- Also: executeUpdate() For INSERT, UPDATE, DELETE, CREATE

API Calls - JDBC

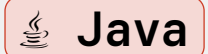
```
1  while(cursor.next()) {  
2      // position in cursor starts at 1 !  
3      string s1 = cursor.getString(1) ;  
4      int i2 = cursor.getInt (2) ;  
5      System.out.println (s1) ;  
6      System.out.println (i2) ;  
7  }  
8  while(myRes.next()) {  
9      System.out.println(i+". Person:  
      "+myRes.getString("fname"));i++;  
10 }
```



API Calls - JDBC

- Important!
- Connections, Statements, ResultSets, etc. hold resources
- Both locally and on the server!
- So: `close()` them as soon as possible
 - ▶ After an error, too!

```
1  finally {  
2      cursor.close () ;  
3      st.close () ;  
4      conn.close () ;  
5  }
```



API Calls - JDBC

How to build a SQL statement programmatically?

API Calls - JDBC

- Problem: use parameters in SQL query

```
1 Statement st = conn.createStatement();  
2 String query = "SELECT id FROM books WHERE title = '" + name +  
  "';";
```

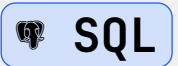
 **Java**

- Problem 1:
 - ▶ name = "0'Reilly";

API Calls - SQL Injection

- If there is nothing to prevent a user from entering “wrong” input, the user can enter some “smart” input like this:

```
1  SELECT UserId , Name , Password FROM Users WHERE UserId =  
   105 OR 1=1;
```

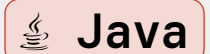


- Problem 2:
 - ▶ SQL injection attacks

API Calls

- One possible Solution: use PreparedStatement

```
1      string name = "O'Reilly" ;
2      string query = "SELECT id FROM tab WHERE name=?" ;
3          // no quotes ( ' ' ) here !
4      PreparedStatement pst = conn.prepareStatement(query) ;
5      pst.setString (1 , name) ;
6      ResultSet cursor = pst.executeQuery () ;
```



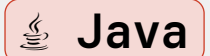
API Calls

- Classes/Interfaces in package `java.sql.*`
 - ▶ `DriverManager`
 - ▶ `Connection`
 - `DatabaseMetaData`
 - `Statement`, `PreparedStatement`, `CallableStatement`
 - `ResultSet`
 - `ResultSetMetaData`
 - `SQLException` (for error handling)

API Calls - Transaction Handling

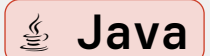
- Transaction syntax:

```
1 connection.setAutoCommit(false);  
2 connection.commit();  
3 connection.rollback();
```



- If you need to change the isolation level, here is the syntax:

```
1 connection.setTransactionIsolation(level);
```



API Calls - Error Handling

- `java.sql.SQLException`
- `getMessage()`: retrieve error text
- `getStatus()`: XOPEN or SQL status
- `getErrorCode()`: vendor-specific error code
- Problem: application needs to know vendor's error codes!
 - ▶ Problem with connection to DB
 - ▶ SQL syntax wrong
 - ▶ Constraint violation
 - ▶ ...

API Calls - Antipatterns

- Do not build SQL string using user input!

```
1  Statement st = conn.createStatement();  
2  String query = "SELECT id FROM books WHERE title = '" + name + "'";
```

 **Java**

- Problems
 - ▶ Correct quoting
 - ▶ Need to handle special characters like '&'
 - ▶ Opens the door for SQL injection attacks

! Memorize

Always use PreparedStatement / parameter binding!

API Calls - Antipatterns

- Do not read whole `ResultSet` into RAM
 - ▶ Problem: `ResultSet` can get huge
 - ▶ Solution: Iterate through the `ResultSet`
- Do not forget to `close()` resources
 - ▶ Problem: Resources are held on client and server!
- Do not implement selection in client code
 - ▶ Problem: `ResultSet` can get huge
 - ▶ Solution: Use `WHERE` clause in SQL

API Calls - Beyond JDBC

- Frameworks on top of JDBC
- spring-jdbc
- Object-Relational Mapping (ORM)
 - ▶ Hibernate
 - www.hibernate.org
 - Mapping is defined in XML configuration files
 - < one - to - many >, < many - to - many >, ...
 - Can generate DDL out of classes + mapping
 - ▶ Different approach: Conventions
 - used by Ruby on Rails (non-Java)

Extend SQL

- The previous approaches of connecting programming languages with DBMS are very fine granular (only one operation at a time)
- Problem: The DBMS cannot optimize because it doesn't know which operation is next
- Base idea: **Extend SQL by control structures**
 - ▶ Putting the application code at the DBMS not at the programming language
- SQL-extensions were former DBMS-specific and called "Stored Procedure"
- Now they are standardized in SQL-99 and called SQL/PSM (persistently stored modules) and therefore over different DBMS useable (e.g., PL/SQL for Oracle)

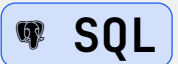
Extend SQL - PL/SQL

- To structure the PL/SQL programs, it's possible to define **procedures** and **functions** and reuse them
 - ▶ A procedure uses parameters like OUT or IN OUT parameters to get the results A procedure may return one or more values through parameters or may not return at all
 - ▶ A function must return a value (of any type) by default definition
- Function can be used in SQL statements, procedures cannot be used in SQL statements

Extend SQL - PL/SQL

- Syntax for creating procedures:

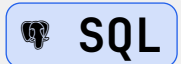
```
1 CREATE PROCEDURE procedure_name
2     ( parameter1 IN parameter_type1 ,
3     ( parameter2 OUT parameter_type2 ,
4     ...
5     ( parameterN IN OUT parameter_typeN) IS
6     <PL/SQL-Block>
```



Extend SQL - PL/SQL

- Syntax for creating functions:

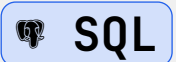
```
1 CREATE FUNCTION function_name
2 (parameter1 parameter_type1 ,
3 (parameter2 parameter_type2 ,
4 ...
5 (parameterN parameter_typeN )
6 RETURN result_type IS
7 <PL/SQL-Block>
```



Extend SQL - PL/SQL

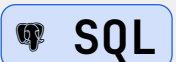
- Using variables and defining data types

```
1 declare
2 today date;
3 type PersonRecordType is record ( PersonName varchar2 ( 50 ) ;
  BirthDate date ) ;
4 employee PersonRecordType;
```



- Cursor for processing results:

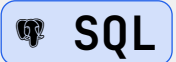
```
1 cursor CurBook is
2 SELECT isbn , title FROM Books ;
```



Extend SQL - PL/SQL

- As control flow structures PL/SQL provides
 - ▶ sequence (by “;”)
 - ▶ condition (where the else branch is optional)

```
1  if <condition > then
2  < PL/SQ-operation >
3  else
4  <PL/SQL-operation>
5  end if ;
```



Extend SQL - PL/SQL

- Loops (for, while, loop)

```
1 while < condition >  
2 loop  
3   < PL/SQL-operation >  
4 end loop ;
```

 SQL

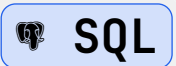
- Executing a relation with infinite loop

```
1 loop  
2   fetch Book into BookRecord;  
3   exit when Book%not found ;  
4   ...  
5 end loop ;
```

 SQL

Extend SQL - Example

```
1 delimiter |
2 CREATE PROCEDURE IF NOT EXISTS
3     output(in ssn char(9), in old_sal DECIMAL(10,2),
4         in new_sal DECIMAL(10,2), in diff_sal DECIMAL(10,2))
5 BEGIN
6     INSERT INTO EMPLOYEE_SALDIFF VALUES
7         ( ssn , old_sal , new_sal, diff_sal);
8 END|
9 delimiter ;
10
11 CALL output(123456789, 12.34, 56.78, 44.44);
```



Extend SQL - Example

```
1      delimiter |
2      CREATE TRIGGER IF NOT EXISTS Print_salary_changes
3      BEFORE UPDATE ON EMPLOYEE
4      FOR EACH ROW
5          BEGIN
6              DECLARE sal_diff DECIMAL(10,2);
7              IF (NEW.salary != OLD.salary)
8                  THEN
9                      BEGIN
10                         SET sal_diff = NEW.salary - OLD.salary ;
11                         CALL output(NEW.ssn, OLD.salary, NEW.salary, sal_diff);
12                     END;
13             END IF;
14      END; | delimiter ;
```



Extend SQL - PL/SQL

- Additional to the structuring, functions/procedures have more advantages:
 - ▶ DBMS can optimize the code because it knows the structure
 - ▶ The execution takes place on the DBMS-server, so network overhead is minimized, which is especially useful in distributed environments (client/server or internet)
 - ▶ Assignment of permissions are available for procedures
 - ▶ Procedures can be used to full integrity constraints

Extend SQL - PL/SQL

- Disadvantages
 - ▶ Software development environments (IDE) are often not optimal
 - ▶ Raised dependency on DBMS
 - ▶ Problems on scalability, because application code is executed on DBS-server instead of being executed by many clients or application servers

Extend SQL - Java Stored Procedures

- Formulating Stored procedures in Java is possible in many DBMS
- Oracle supports the execution of Java programs directly on the server
- Java programs with GUI are excluded
- Access by wrapping Java methods in PL/SQL
- The mapping of PL/SQL call on Java method must be created by the programmer
- These mapped Java methods can be accessed by all DML operations (Select, Update, Insert, Delete) and within PL/SQL blocks

3.2 Summary

- Embed SQL commands into host language
 - ▶ Advantages +
 - Query is part of source code
 - syntax checking
 - validation against the database schema
 - readable
 - Disadvantages -
 - Static queries
 - Changes of queries go through recompilation process

3.2 Summary

- SQL commands through API calls
 - ▶ Advantages +
 - More flexibility
 - Queries can be generated at runtime
 - Disadvantages -
 - More complex programming
 - No checking during compile time

3.2 Summary

- Extend SQL
 - ▶ Advantages +
 - No suffering from impedance mismatch problem
 - Disadvantages -
 - New language for the programmer

4. License Notice

4.1 Attribution

The basis of this course stems from: Professor Dr. Ulrike Herster The following license applies to all unmodified and modified material used throughout this course.

! License

The publication or sharing of slides, image and sound recordings of this course are not permitted.

© Professor Dr. Ulrike Herster

This video and the presented script are protected by copyright. The use is only permitted in relation with the course of study. It is not permitted to forward or republish it in other places (especially in the internet).